SIMATS ENGINEERING

CO2_ASSESSMENT TOOL3_ Dry Run Evaluation

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1709**

Register Number	**192312428**
Name	**B.BHAVANA**

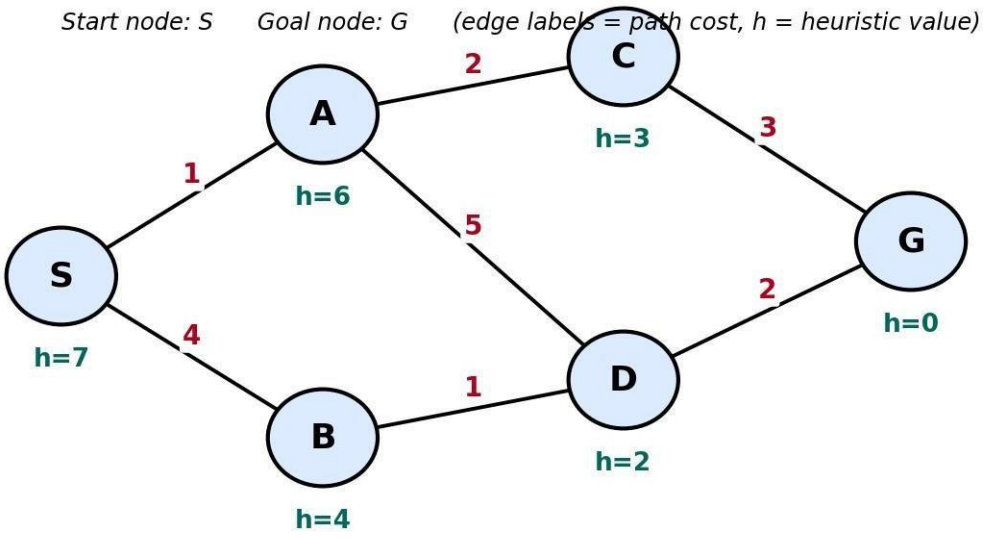
COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 20%

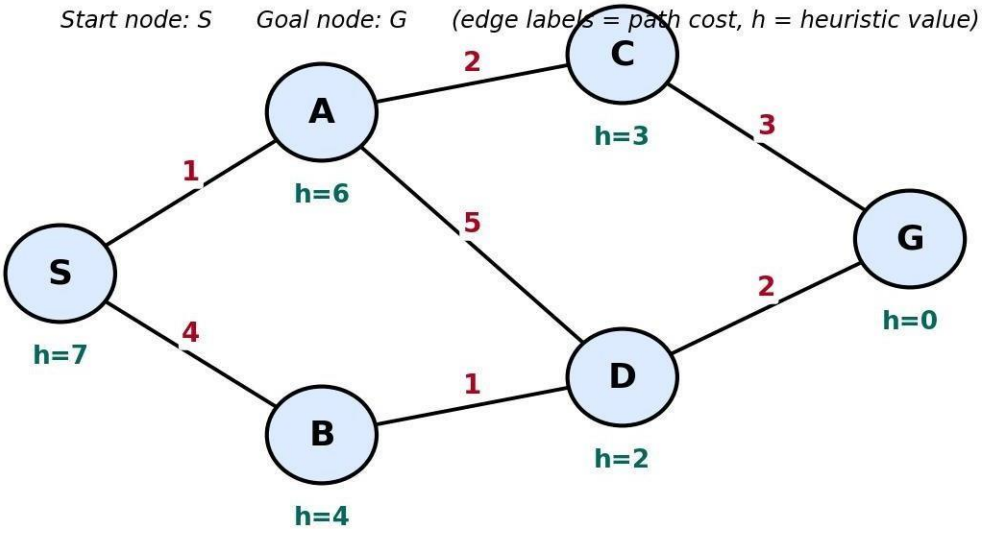
QUESTIONS: 5 **Total Marks:50**

Q1. Consider a suitable state-space graph (with heuristic values h(n) assigned to each node) of your choice for a given problem. Perform a dry run of Greedy Best-First Search from the start node to the goal node. Clearly show the frontier/open list, the node selected for expansion at each step, and the final path obtained.

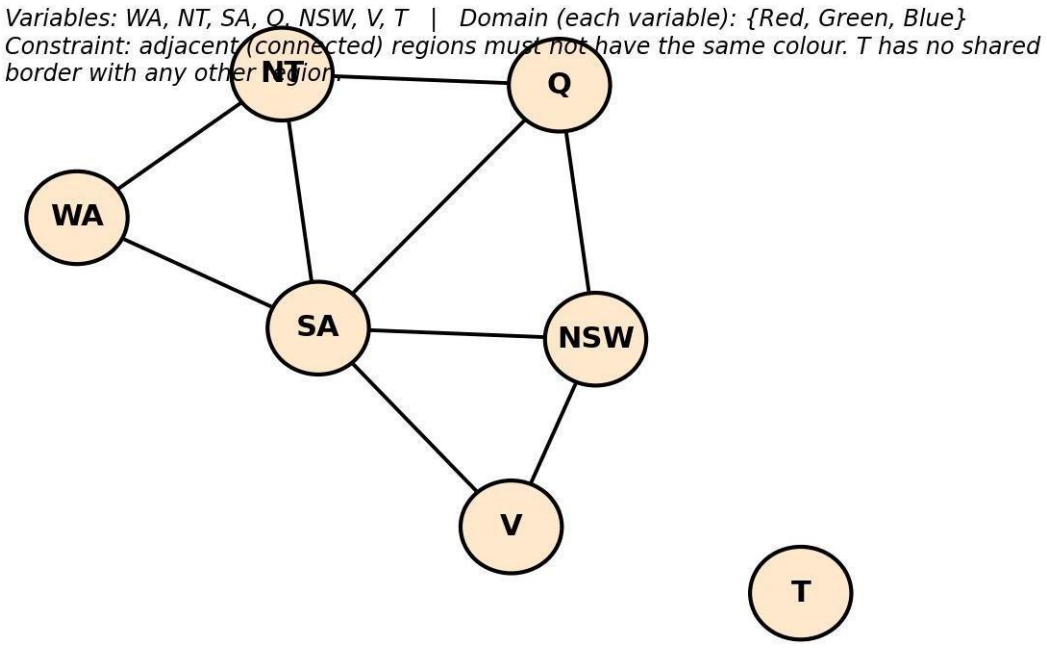


Q2. For a suitable weighted state-space graph (with edge costs and heuristic values h(n)) of your choice, perform a dry run of A* Search from the start node to the goal

node. Show the g(n), h(n) and f(n) values computed at every step, the order in which nodes are expanded, and the final optimal path along with its total cost.

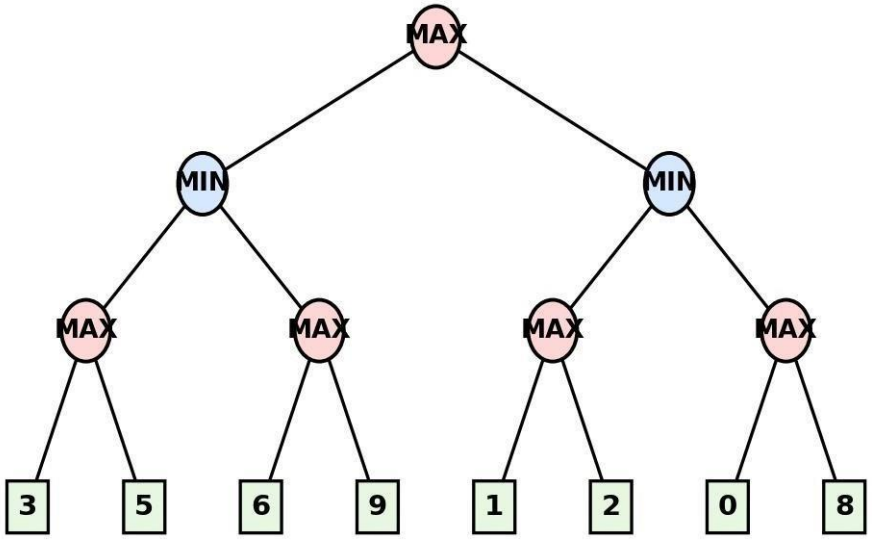


Q3. Formulate a given Constraint Satisfaction Problem (e.g., Map Colouring / N-Queens) by clearly stating the variables, their domains, and the constraints involved. Perform a dry run of the Backtracking Search algorithm, showing each variable assignment, every constraint check, and any backtracking steps, until a complete solution (or failure) is reached.



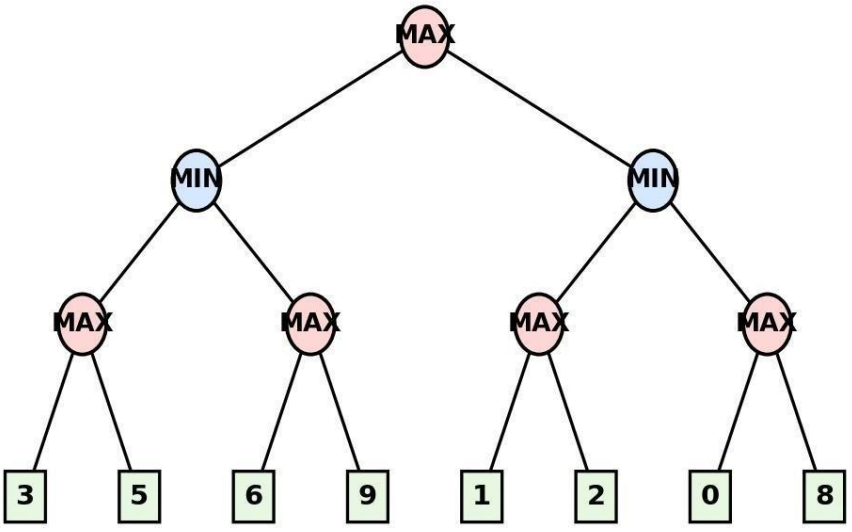
Q4. For a given two-player game tree of your choice (at least 3 levels deep), perform a dry run of the Minimax algorithm. Show the utility values at the leaf/terminal nodes and the backed-up values computed at every internal node, and state the best move for the MAX player with justification.

Root = MAX to move. Squares at the bottom are terminal (leaf) nodes with their utility values.



Q5. Using the same (or a different) game tree from Q4, perform a dry run of the Minimax algorithm with Alpha-Beta Pruning. Show the α and β values maintained at each node during the traversal, clearly indicate which branches get pruned and why, and state the final best move obtained.

Root = MAX to move. Squares at the bottom are terminal (leaf) nodes with their utility values.



Code of Conduct:

I B.Bhavana(192312428) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student

Dry Run Evaluation

RUBRICS FOR CALCULATION

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Max Marks
Problem Formulation & Setup (states/nodes, heuristic values or CSP variables, domains and constraints correctly identified)	Accurately identifies all required elements (states, heuristics/costs, or variables/domains/constraints) with clear, correct notation.	Identifies most required elements correctly, with only minor omissions or notational errors that do not affect the dry run.	Identifies some required elements; noticeable errors or missing components affect the later steps.	Little or no correct problem formulation; required elements are largely missing or incorrect.	**10**
Correct Application of Algorithm Procedure (expansion order, backtracking, or pruning as per the standard method)	Follows the exact algorithmic procedure at every step with no deviation from the standard method.	Follows the procedure correctly for most steps, with only one or two minor deviations that do not change the outcome.	Several steps deviate from the correct procedure, though the overall approach is recognisable.	Algorithm steps are largely incorrect, or the procedure is not followed.	**10**
Accuracy of Computations (g(n), h(n), f(n) values, minimax backed-up values, or α–β updates)	All computed values are accurate and consistent throughout the dry run.	Minor calculation errors are present but do not significantly affect the final outcome.	Multiple calculation errors are present that affect the correctness of intermediate steps.	Calculations are mostly incorrect or missing.	**10**

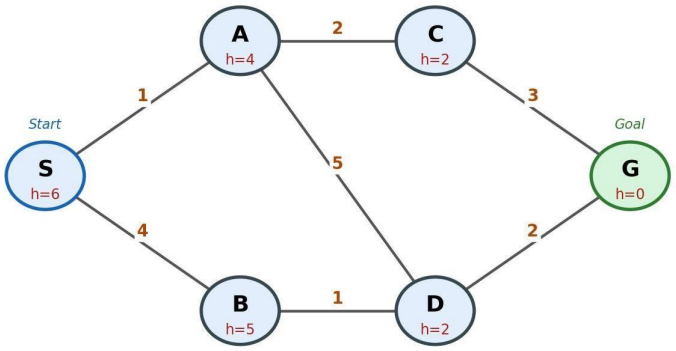
Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Max Marks
Correctness of Final Outcome (final path, CSP solution, or best move obtained)	Final result is completely correct and matches the expected optimal outcome.	Final result is mostly correct, with only a minor deviation from the optimal outcome.	Final result is only partially correct.	Final result is incorrect or not obtained.	**10**
Clarity of Presentation & Justification (trace tables/trees/ diagrams, explanation of each step)	Dry run is presented clearly using well-organised tables/trees/di agrams, with every step explained and justified.	Dry run is reasonably clear and mostly organised, with minor gaps in explanation .	Dry run presentation is unclear or poorly organised, with limited justification.	Dry run is disorganised or illegible, with little to no justification.	**10**
				Total	50

Q1. Greedy Best-First Search — Dry Run

Problem Setup

Consider the following state-space graph with start node S and goal node G. Each node carries a heuristic estimate h(n) — the straight-line/estimated distance to the goal. Greedy Best-First Search always expands the node in the frontier with the lowest h(n), ignoring the path cost already incurred.

State-Space Graph with Edge Costs g(n) and Heuristic Values h(n)



Heuristic table (estimated distance to goal G):

Node	*S*	*A*	*B*	*C*	*D*	*G*
h(n)	*6*	*4*	*5*	*2*	*2*	*0*

Dry Run Trace

At every step, the node with the smallest h(n) in the frontier is selected for expansion. Ties are broken alphabetically.

Step	*Node Expanded*	*Frontier (Open List) after expansion — sorted by h(n)*	*Goal Reached?*
1	*S (h=6)*	*A(4), B(5)*	*No*
2	*A (h=4)*	*C(2), D(2), B(5)*	*No*
3	*C (h=2)*	*G(0), D(2), B(5)*	*No*
4	*G (h=0)*	*— (goal popped from frontier)*	*Yes — STOP*

Result

Final path: S → A → C → G

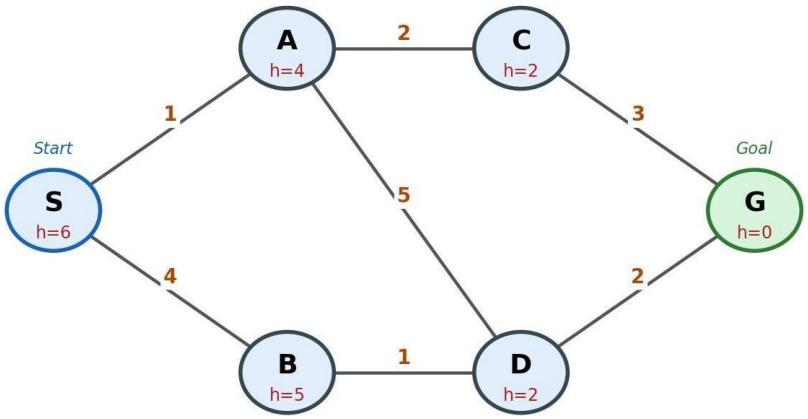
Path cost (for reference only — Greedy Search does not use this value to guide the search): g(S→A) +g(A→C) +g(C→G) = 1 + 2 + 3 = 6.

Q2. A* Search — Dry Run

Problem Setup

We reuse the same weighted state-space graph (edge costs and heuristic values shown below) so the two searches can be directly compared. A expands the node with the lowest f(n) = g(n) + h(n), where g(n) is the true cost from the start accumulated so far and h(n) is the heuristic estimate to the goal.*

State-Space Graph with Edge Costs g(n) and Heuristic Values h(n)



Edge costs: S–A=1, S–B=4, A–C=2, A–D=5, B–D=1, C–G=3, D–G=2

Heuristics: h(S)=6, h(A)=4, h(B)=5, h(C)=2, h(D)=2, h(G)=0

Dry Run Trace — g(n), h(n), f(n) at every step

Step	***Node Expanded***	***Children Generated → g(n), h(n), f(n)=g+h***	***Frontier after step (sorted by f)***
1	*S (g=0,h=6,f=6)*	*A: g=1, h=4, f=5* *B: g=4, h=5, f=9*	*A(5), B(9)*
2	*A (g=1,h=4,f=5)*	*C: g=3, h=2, f=5* *D: g=6, h=2, f=8*	*C(5), D(8), B(9)*
3	*C (g=3,h=2,f=5)*	*G: g=6, h=0, f=6*	*G(6), D(8), B(9)*
4	*G (g=6,h=0,f=6)*	*Goal test passed — lowest f(n) in frontier and it is the goal node*	*STOP*

Result

Optimal path: S → A → C → G

Total optimal path cost g(G) = 1 + 2 + 3 = 6.

Optimality check: the only alternative complete path is S → B → D → G, with cost 4 + 1 + 2 = 7. Since 6 < 7, A* has correctly returned the least-cost path — confirming optimality, which holds here because h(n) is admissible (never overestimates the true remaining cost, e.g. h(C)=2 ≤ actual cost 3 to G).

Question 3: CSP Formulation and Backtracking Search Dry Run

A Constraint Satisfaction Problem (CSP) consists of variables, domains, and constraints. The objective is to assign values to all variables such that every constraint is satisfied. The Backtracking Search algorithm assigns values sequentially and backtracks whenever a constraint is violated.

Problem Formulation

Variables

{WA, NT,SA,Q,NSW,V,T}
Domain

Each region can be assigned one of the following colours:
{Red,Green,Blue}

Constraints

Adjacent regions must have different colours.

WA → NT, SA

NT → WA, SA, Q

SA → WA, NT, Q, NSW, V

Q → NT, SA, NSW

NSW → SA, Q, V

V → SA, NSW

T → No adjacent regions

Backtracking Search (Dry Run)

Step	Assignment	Constraint Check
1	WA = Red	No conflict
2	NT = Red	Same as WA
	NT = Green	Valid
3	SA = Red	Same as WA
	SA = Green	Same as NT
	SA = Blue	Valid
4	Q = Red	Different from NT and SA
5	NSW = Red	Same as Q
	NSW = Green	Valid
6	V = Red	Different from SA and NSW
7	T = Red	No adjacent regions

Final Solution

Region	Colour
WA	Red
NT	Green
SA	Blue
Q	Red
NSW	Green
V	Red
T	Red

The Australia Map Colouring problem is solved using the **Backtracking Search** algorithm. The algorithm assigns colours one by one, checks all constraints, and backtracks whenever a conflict occurs.

Question 4: Minimax Algorithm - Dry Run

Game tree used (matching the figure): Root = MAX to move, 3 levels deep, with terminal (leaf) utility values, read left to right, of: 3, 5, 6, 9, 2, 2, 0, 3.

The Minimax algorithm is an AI search algorithm used in two-player, zero-sum games. It assumes that MAX tries to maximize the utility value, while MIN tries to minimize it. The algorithm evaluates the game tree from the leaf nodes upward to determine the optimal move.

Game Tree

Leaf node utility values:

Left Subtree: (3, 5), (6, 9)

Right Subtree: (1, 2), (0, 8)

Step 1: Evaluate MAX Nodes

MAX Node	Leaf Values	Selected Value
MAX_1	3, 5	5
MAX_2	6, 9	9
MAX_3	1, 2	2
MAX_4	0, 8	8

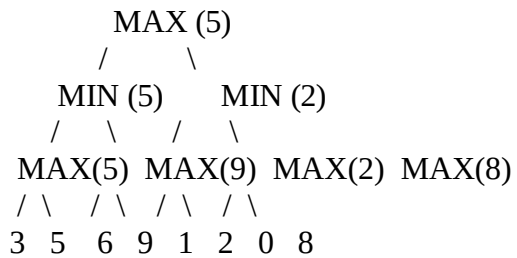
Step 2: Evaluate MIN Nodes

MIN Node	Children Values	Selected Value
Left MIN	5, 9	5
Right MIN	2, 8	2

Step 3: Evaluate Root MAX Node

Root MAX Children Values Selected Value
MAX 5, 2 5

Backed-Up Game Tree



Dry Run Summary

Step	Operation	Result
1	MAX(3,5)	5
2	MAX(6,9)	9
3	MIN(5,9)	5
4	MAX(1,2)	2
5	MAX(0,8)	8
6	MIN(2,8)	2
7	MAX(5,2)	5

The MAX player chooses the left subtree because it guarantees the highest utility value (5) after considering the optimal moves of the MIN player.

The Minimax algorithm evaluates the game tree from the leaf nodes to the root. The optimal utility value obtained is 5, so the left branch is the best move for the MAX player.

Q5. Minimax with Alpha-Beta Pruning — Dry Run

Alpha–Beta Pruning is an optimization technique for the Minimax algorithm. It reduces the number of nodes evaluated by pruning branches that cannot affect the final decision. Here:

α (Alpha): Best value MAX can guarantee.

β (Beta): Best value MIN can guarantee.

A branch is pruned when $\alpha \geq \beta$.

Game Tree

Leaf node utility values:

Left Subtree: (3, 5), (6, 9)

Right Subtree: (1, 2), (0, 8)

Traversal is performed from left to right.

Dry Run with Alpha–Beta Pruning

Step 1: Root (MAX)

$\alpha = -\infty$

$\beta = +\infty$

Explore the left subtree.

Step 2: Left MIN Node

$\alpha = -\infty$

$\beta = +\infty$

First MAX Node

Leaf values: 3, 5

MAX = 5

MIN updates:

$\beta = \min(+\infty, 5) = 5$

Second MAX Node

Current values:

$\alpha = -\infty$

$\beta = 5$

Evaluate first leaf:

Leaf = 6

MAX updates:

$\alpha = \max(-\infty, 6) = 6$

Since

$\alpha\ (6) \geq \beta\ (5)$

➞ Prune the next leaf (9).

MAX returns 6.

MIN chooses

min(5,6) = 5

Return 5 to the root.

Root updates:

$\alpha = \max(-\infty, 5) = 5$

Step 3: Right MIN Node

Current values:

$\alpha = 5$

$\beta = +\infty$

First MAX Node

Leaf values:

1,2

MAX = 2

MIN updates:

$\beta = \min(+\infty, 2) = 2$

Now,

$\alpha (5) \geq \beta (2)$

➞ Prune the entire right MAX subtree (0,8).

MIN returns 2.

Step 4: Root MAX

Receives

Left = 5

Right = 2

MAX selects

max(5,2) = 5

Alpha–Beta Values

Node	α	β	Value Returned
Root MAX	5	+∞	5
Left MIN	−∞	5	5
Left MAX	5	+∞	5
Second MAX	6	5	6 (Pruned)
Right MIN	5	2	2
Third MAX	2	+∞	2
Fourth MAX	Pruned	Pruned	Not Evaluated

Pruned Branches

Pruned Branch	Reason
Leaf node 9	$\alpha = 6 \geq \beta = 5$
Entire subtree (0, 8)	$\alpha = 5 \geq \beta = 2$

Final Result

Root Values	Best Value
Left = 5	5
Right = 2	

Best Move for MAX: Left Subtree

Using Alpha–Beta Pruning, unnecessary branches (9) and (0, 8) are pruned because they cannot influence the final decision ($\alpha \geq \beta$). This reduces the number of nodes evaluated while producing the same optimal result as the Minimax algorithm. Therefore, the best move for the MAX player is the left subtree with a utility value of 5.